## 1 표지 — 트랙 개시 · 관리




**발표 대본** 표지 · 약 25초

안녕하세요. 오늘은 트랙 개시와 트랙 관리에 대해 이야기해 보겠습니다.

지난 과제에서는 표적 참값에 잡음을 더해 플롯을 만들고 클러터도 같이 만들어 봤는데요. 이번에는 그 플롯을 데이터 처리 쪽으로 보내서, 같은 표적에서 나온 것 같은 플롯 두 개를 이어 트랙을 시작하고, 그렇게 만든 트랙을 점수로 관리하는 부분까지 만들어 봤습니다.

## 2 목차

INDEX

트랙 개시 · 관리
TrackMgr

Janghwan Kim (Harley)


00 과제 개요 3

01 트랙 개시 4

02 트랙 관리 10

03 구현 15

04 결과 23


2

**발표 대본** 목차 · 약 30초

먼저 이번 과제가 무엇을 요구하는지 한 장으로 정리하고 시작하겠습니다.

그다음 트랙 개시에서는 플롯 한 개로 시작하는 방법과 두 개로 시작하는 방법을 비교하고, 이번에 쓴 2-Point 개시의 계산과 속도 게이트를 보겠습니다. 이어서 트랙 관리에서는 트랙이 어떤 상태를 거치는지 보고, M/N 방식과 트랙 스코어 방식을 비교해 보겠습니다.

그 뒤에 이 내용을 코드로 어떻게 구현했는지 보고, 마지막으로 실제 프로그램을 돌린 결과를 같이 보겠습니다.

# 3 이번 과제 한눈에 보기



00. 과제 개요 과제 1 · 2 · 3

## 이번 과제 한눈에 보기

레이다가 잰 점 (플롯) 을 이어 표적의 움직임 (트랙) 을 처음 만들고, 그 트랙을 점수로 관리하는 기능을 구현했습니다.

**이번 과제에서 하는 일 세 가지**

① 트랙 개시
연속 두 스캔의 플롯 두 개로 트랙을 만든다
2-Point Track Initiation, 속도 게이트 800 m/s

② 트랙 관리
트랙마다 점수를 매겨 확정 · 메모리 · 삭제를 정한다
트랙 스코어 방식, 문턱 5 / 0 / 5

③ 시뮬레이션
플롯 메시지를 UDP 로 보내고 받아 처리한다
2D · 3D 화면에 참값 · 플롯 · 트랙을 함께 표시

**과제 요구사항과 구현한 곳** (오른쪽 끝은 이 자료의 장)

| 과제 항목 | 요구 사항 | 구현 (소스) | 장 |
|---|---|---|---|
| 트랙 개시 방식 | 2 Point Track Init. | f_Trk_TwoPointInit (TrackMgr.c) | 1.2 ~ 1.3, 3.6 |
| 상태변수 | [x x' x'' y y' y'' z z' z'']  9 개, 가속도 0 | mat_EstX 9 x 1,  mat_EstP 9 x 9 | 1.3, 3.6 |
| 속도 게이팅 | V_max = 800 m/s | f_Trk_ProcessScan | 1.4, 3.5 |
| 트랙 ID | 100 ~ 10000 | f_Trk_NewId | 3.5 |
| 트랙 관리 | 트랙 스코어 방식, 문턱 5 / 0 / 5 | f_Trk_UpdateState | 2.3, 3.7 |
| 플롯 메시지 | STRUCT_PlotMsg, 안테나 구좌표 | Idd.h, 송신 · 수신 쓰레드 (UDP) | 3.1 ~ 3.4 |
| 플롯 ID | 10000 ~ 1000000, 표적 + 클러터 | f_Lnk_BuildMsg (PlotLink.c) | 3.3 |
| 트랙 연관 | Assigned_Flag 매 스캔 0 (실패) | f_Trk_ProcessScan | 2.1, 3.5 |
| 결과 화면 | True · Plot · Track 심볼 2D · 3D | TargetSimUI (MFC) | 4.1 ~ 4.3 |

3

## 발표 대본

과제 개요 · 약 60초

이번 과제는 크게 세 가지입니다. 왼쪽을 보시면, 첫째는 트랙 개시입니다. 연속된 두 스캔에서 들어온 플롯 두 개를 이어서 트랙을 처음 만드는 건데, 두 플롯이 같은 표적인지 거르기 위해 속도 게이트 800 m/s 를 씁니다. 둘째는 트랙 관리입니다. 만든 트랙마다 점수를 매겨서 확정할지, 잠시 기다릴지, 지울지를 정합니다. 셋째는 시뮬레이션입니다. 플롯을 메시지로 만들어 UDP 로 보내고 받는 구조를 만들고, 결과를 2D 와 3D 화면에 참값, 플롯, 트랙을 함께 그렸습니다.

오른쪽 표는 과제 문서의 요구사항을 하나씩 적고, 그걸 어느 함수에서 구현했는지, 이 자료의 어느 장에서 설명하는지 적어 둔 것입니다. 발표 중에 "이건 과제의 어느 조건이지?" 싶으시면 이 표를 참고해 주시면 됩니다.

## 01. 트랙 개시

트랙이 무엇인지부터 보고, 플롯 두 개로 트랙을 시작하는 2-Point 개시와 속도 게이트를 설명합니다.


**1.1** 트랙이란 ..... 5

**1.2** 1-Point 와 2-Point 개시 ..... 6

**1.3** 2-Point 개시의 계산 ..... 7

**1.4** 속도 게이팅 ..... 8

**1.5** 측정 오차를 VC-NED 로 옮기기 ..... 9




**발표 대본** 트랙 개시 · 약 15초

첫 번째로 트랙 개시입니다. 트랙이 무엇인지부터 짚고, 1-Point 와 2-Point 개시, 2-Point 의 계산, 속도 게이트, 그리고 측정 오차를 트랙 좌표계로 옮기는 부분 순서로 가 보겠습니다.

# 5 1.1 트랙이란




## 발표 대본

트랙 개시 · 약 60초

먼저 이번 과제가 전체 흐름에서 어디쯤인지부터 보겠습니다. 위쪽을 보면, 과제 3 에서 표적이 실제로 움직인 궤적을 만들었고, 과제 4 에서 그 궤적을 레이다가 잰 것처럼 플롯과 클러터로 바꿨습니다. 이번 과제 5 는 그 플롯을 받아서 트랙을 시작하고 관리하는 부분이고, 트랙에 다음 플롯을 이어 붙이는 연관과 추적 필터는 다음 과제에서 다룹니다.

아래 그림을 보시면 레이다는 0.1 초마다 한 번씩 전체를 훑는데, 이걸 스캔이라고 부릅니다. 스캔마다 플롯이 여러 개 들어오는데, 이 중에는 진짜 표적 플롯도 있고 클러터도 섞여 있습니다. 레이다 입장에서는 둘 다 그냥 점입니다. 여기서 같은 표적에서 나온 점을 시간 순서대로 이어서 표적의 위치와 속도를 추정한 것을 트랙이라고 합니다. 트랙을 처음 만드는 것을 트랙 개시, 만든 트랙이 진짜 표적인지 판단해서 확정하거나 지우는 것을 트랙 관리라고 부릅니다.

오른쪽에는 오늘 계속 나올 용어 다섯 개를 정리해 두었으니 참고해 주세요.

# 1.2 1-Point 와 2-Point 개시




## 발표 대본

트랙 개시 · 약 60초

트랙을 시작하는 방법은 크게 두 가지가 있습니다.

왼쪽의 1-Point 개시는 플롯 하나만 가지고 바로 트랙을 만드는 방법이에요. 위치는 플롯 위치를 그대로 쓰면 되는데 문제는 속도입니다. 점 하나만 봐서는 표적이 어느 쪽으로 가는지 알 수가 없거든요. 그래서 속도는 0 으로 두고 오차를 크게 잡습니다. 빨리 시작할 수 있는 대신 초기 속도 오차가 크다는 게 단점입니다.

오른쪽의 2-Point 개시는 연속된 두 스캔, 또는 두 드웰에서 받은 플롯을 씁니다. 두 플롯 사이의 위치 변화를 시간으로 나누면 속도가 나오니까 속도 오차를 줄일 수 있습니다. 대신 플롯이 두 번 들어올 때까지 기다려야 해서 개시가 한 스캔 늦어집니다. 오른쪽 표에 두 방식의 차이를 정리해 두었습니다.

이번 과제는 과제 조건대로 2-Point 개시를 적용했고, 두 플롯이 정말 같은 표적인지는 점선 원으로 그린 속도 게이트로 먼저 걸러 냅니다.

# 1.3 2-Point 개시의 계산

**01. 트랙 개시** 과제 1.2 · 3-1

## 1.3 2-Point 개시의 계산: 초기 상태변수와 오차 공분산

트랙은 상태변수 X (위치 · 속도 · 가속도) 와 그 값이 얼마나 틀릴 수 있는지 적은 오차 공분산 P 를 갖습니다. 둘 다 플롯 두 개로 계산합니다.

**1 상태변수 X** 세 축 (x, y, z) 마다 위치 · 속도 · 가속도 = 9 개

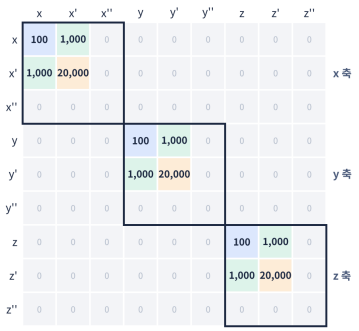
예 (x 축) 앞 스캔 플롯 5,000 m, 이번 스캔 플롯 5,030 m, 스캔 간격 T = 0.1 s

| 성분 | 식 | 예 (x 축) |
|---|---|---|
| 위치 x | z_k (이번 플롯 그대로) | 5,030 m |
| 속도 x' | ( z_k − z_k-1 ) / T | ( 5,030 − 5,000 ) / 0.1 = 300 m/s |
| 가속도 x'' | 0 (과제 조건) | 0 |

**2 오차 공분산 P** 한 축의 3 x 3 (예: 플롯 오차 ±10 m, T = 0.1 s)

| | 위치 | 속도 | 가속도 |
|---|---|---|---|
| **위치** = 이번 플롯 | **R_k** 100 (±10 m) | **R_k / T** 1,000 | 0 |
| **속도** = 두 플롯 차 / T | **R_k / T** 1,000 | **( R_k + R_k-1 ) / T²** 20,000 (±141 m/s) | 0 |
| **가속도** = 0 | 0 | 0 | 0 |

**3 9 x 9 전체 P** ② 의 3 x 3 블록이 축마다 대각선에 하나씩




**눈여겨볼 점** 위치 오차 ±10 m 가 0.1 초로 나뉘면서 속도 오차 ±141 m/s 가 됩니다.
R: 플롯 위치의 측정 오차 (분산). 구하는 법은 1.5

7

## 발표 대본

트랙 개시 · 약 75초

2-Point 로 만든 트랙이 어떤 값을 갖는지 숫자로 따라가 보겠습니다.

1 번 상태변수부터 보면, x 축에서 앞 스캔 플롯이 5,000 m, 이번 스캔 플롯이 5,030 m 이고 스캔 간격이 0.1 초라고 해 볼게요. 위치는 이번 플롯 그대로 5,030 m, 속도는 30 m 를 0.1 초로 나눈 300 m/s, 가속도는 과제 조건대로 0 입니다. y, z 축도 같은 식이라 모두 9 개 값이 나옵니다.

2 번 오차 공분산 P 는 이 값들이 얼마나 틀릴 수 있는지를 적은 표인데요, 플롯 오차가 ±10 m 라고 해 보겠습니다. 위치는 이번 플롯 오차가 그대로 들어가니까 분산이 10 의 제곱인 100 입니다. 속도는 두 플롯 차를 0.1 초로 나눈 값이라 두 오차가 더해져서 200 이 되고, 0.1 의 제곱으로 나누면 20,000, 표준편차로는 ±141 m/s 입니다. 위치와 속도에는 둘 다 이번 플롯 오차가 들어 있어서 같이 움직이고, 그 값이 1,000 입니다.

3 번 9 x 9 전체는 이 3 x 3 블록이 축마다 대각선에 하나씩 놓인 모양입니다. 위치 오차 10 m 가 속도에서는 141 m/s 로 커진다는 점은 결과에서 다시 나옵니다.

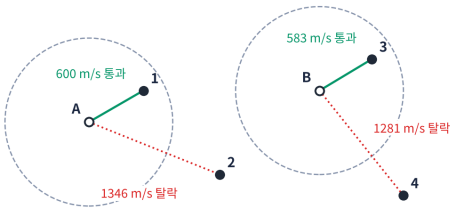
## 1.4 속도 게이팅

표적이 아무리 빨라도 한 스캔 동안 갈 수 있는 거리는 정해져 있습니다. 두 플롯 사이 거리를 시간으로 나눈 속력이 V_max 보다 작을 때만 같은 표적으로 봅니다.

$$| z_{(k,i)} - z_{(k-1,j)} | / \Delta T < V_{max}$$

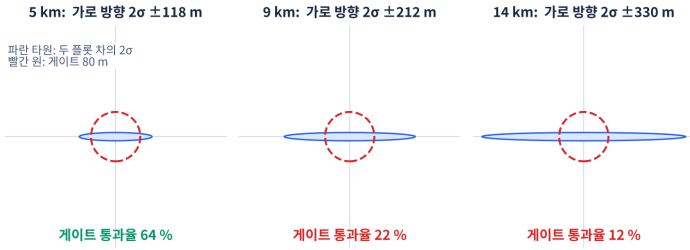
V_max = 800 m/s, ΔT = 0.1 s -> 앞 플롯에서 반경 80 m 안이면 통과
i: 이번 스캔 플롯, j: 앞 스캔 플롯. 거리는 VC-NED 직교좌표 (북 · 동 · 아래) 로 잰다

짝짓기: 앞 스캔 플롯 (빈 원) 과 이번 스캔 플롯 (채운 원)




점선 원 = 반경 V_max · T = 80 m
속력이 작은 쌍부터 하나씩 고르고, 쓴 플롯은 다시 쓰지 않는다

거리별: 같은 표적 두 플롯의 차이 (파란 타원) 와 게이트 (빨간 원)






### 발표 대본

트랙 개시 · 약 75초

이제 속도 게이트입니다. 앞 스캔 플롯과 이번 스캔 플롯이 같은 표적에서 나왔는지 어떻게 알 수 있을까요? 생각보다 간단한데요, 표적이 아무리 빨라도 한 스캔 동안 갈 수 있는 거리는 정해져 있습니다. 그래서 두 플롯 사이 거리를 스캔 간격으로 나눈 속력이 최대 속력 V_max 보다 작으면 같은 표적일 수 있다고 봅니다. 과제 값이 800 m/s 이고 스캔 간격이 0.1 초니까, 앞 플롯에서 반경 80 m 안에 들어와야 통과입니다.

왼쪽 그림을 보면 A 와 1 은 600 m/s 라서 통과, A 와 2 는 1,346 m/s 라서 탈락입니다. 한 플롯이 여러 쌍에 걸릴 수도 있어서, 통과한 쌍 중에 속력이 가장 작은 쌍부터 하나씩 고르고 한 번 쓴 플롯은 다시 쓰지 않습니다.

오른쪽은 거리별로 같은 표적의 두 플롯 차이를 파란 타원으로, 게이트를 빨간 원으로 겹쳐 그린 그림입니다. 각도 오차에 거리를 곱한 만큼이 가로 방향 위치 오차가 되니까, 멀수록 타원이 옆으로 길어집니다. 5 km 에서는 타원이 게이트를 조금 넘는 정도라 열 번에 여섯 번쯤 통과하지만, 14 km 에서는 게이트보다 훨씬 길어서 진짜 표적 쌍인데도 열 번에 한 번 정도만 통과합니다.

# 9 1.5 측정 오차를 VC-NED 로 옮기기




## 발표 대본

트랙 개시 · 약 70초

앞에서 R 이라고 부른 측정 오차를 어떻게 구하는지 보겠습니다. 플롯은 안테나 기준 거리, 방위각, 고각으로 들어오는데 트랙은 VC-NED 좌표계에서 만들기 때문에, 오차도 VC-NED 로 옮겨 줘야 합니다.

VC-NED 는 플랫폼 무게중심이 원점이고 축이 북, 동, 아래로 고정된 좌표계입니다. 플랫폼 자세가 바뀌어도 축이 같이 돌지 않아서 트랙을 만들기에 편합니다.

순서는 이렇습니다. 먼저 거리, 방위각, 고각의 표준편차를 야코비안 J 로 안테나 직교좌표의 오차로 바꾸고, 그다음 안테나에서 동체, 동체에서 VC-NED 로 가는 회전 M 을 앞뒤로 곱합니다.

오른쪽 그림이 대함 표적인 표적 1 플롯의 실제 예입니다. 왼쪽 칸처럼 빔 기준으로 보면 거리 방향 오차는 4.7 m 로 얇고 옆으로는 118 m 로 넓어서, 기울지 않은 납작한 타원이고 R 도 대각선에만 값이 있습니다. 그런데 이 표적은 정북에서 동쪽으로 조금 벗어난 곳에 있어서, 같은 타원을 북 · 동 축으로 보면 오른쪽 칸처럼 비스듬해집니다. 그러면 두 축의 오차가 함께 움직여서 R 의 대각선 밖 칸에도 큰 값이 생깁니다. 이 R 이 1.3 의 오차 공분산에 그대로 들어갑니다.

## 02. 트랙 관리

만든 트랙이 진짜 표적인지 가려 확정하거나 지우는 방법입니다. 트랙의 네 가지 상태와 연관을 보고, M/N 방식과 이번에 쓴 트랙 스코어 방식을 비교합니다.


**2.1** 트랙 상태와 연관 ..... 11

**2.2** M/N 방식 ..... 12

**2.3** 트랙 스코어 방식 ..... 13

**2.4** 점수가 바뀌는 과정 (예) ..... 14




**발표 대본** 트랙 관리 · 약 15초

두 번째로 트랙 관리입니다. 트랙이 어떤 상태를 거치는지와 연관이 무엇인지 먼저 보고, M/N 방식과 이번에 쓴 트랙 스코어 방식, 그리고 점수가 실제로 어떻게 바뀌는지까지 보겠습니다.



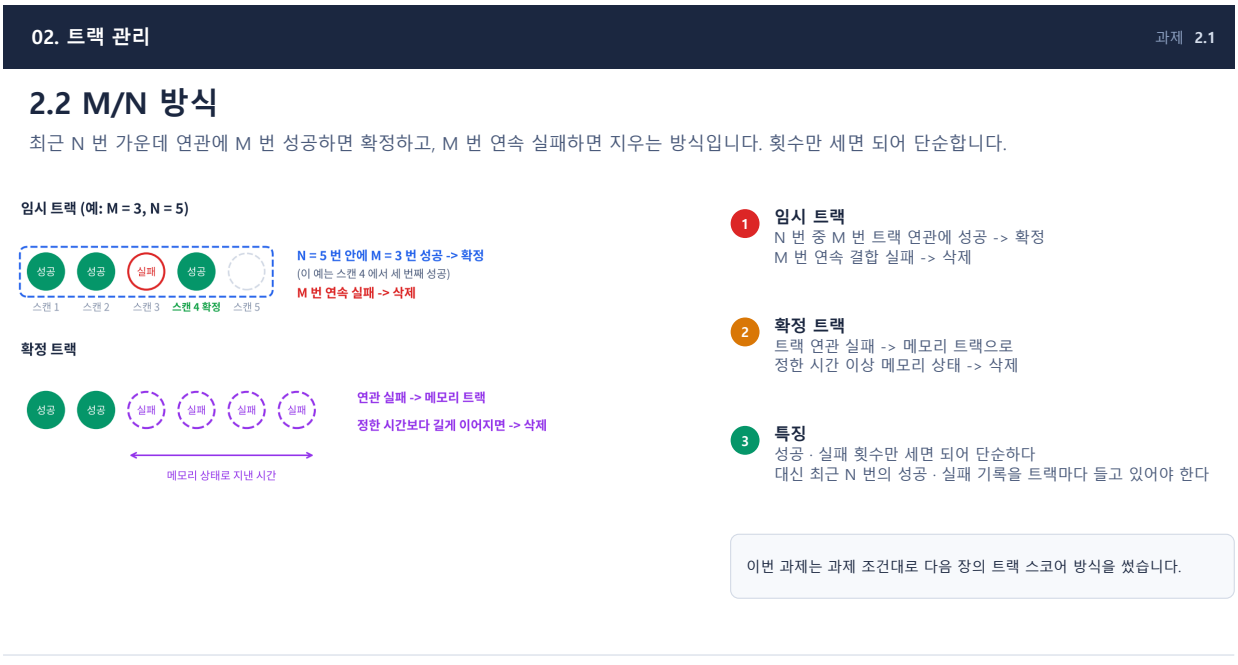

## 발표 대본

트랙 관리 · 약 55초

관리 방법을 보기 전에 용어 두 가지를 먼저 정리하고 가겠습니다.

왼쪽은 연관입니다. 새 스캔이 들어오면 트랙이 있을 법한 자리 근처에 플롯이 들어왔는지 보는데요, 들어와서 그 플롯을 트랙에 이어 붙이면 연관 성공, 아무것도 들어오지 않으면 연관 실패입니다. 이 결과를 트랙마다, 스캔마다 1 과 0 으로 남긴 것이 Assigned_Flag 입니다. 연관 자체는 다음 과제 내용이라 이번에는 과제 조건대로 늘 0 으로 둡니다.

오른쪽은 트랙 상태 네 가지입니다. 막 만든 트랙은 임시 트랙이에요. 클러터 두 개가 우연히 이어진 것일 수도 있으니까 아직 확인하는 중인 상태입니다. 연관이 계속 이어지면 진짜 표적으로 보고 확정 트랙이 됩니다. 확정 트랙이 플롯을 한두 번 놓쳤다고 바로 지우면 아까우니까, 메모리 트랙으로 두고 다시 잡히기를 기다립니다. 그래도 안 잡히거나 임시 트랙이 계속 실패하면 삭제 트랙이 됩니다.




## 발표 대본

트랙 관리 · 약 45초

첫 번째 관리 방법은 M/N 방식입니다. 최근 N 번 중에 연관에 몇 번 성공했는지를 세는 방법이에요.

위쪽 임시 트랙 예시는 M 이 3, N 이 5 입니다. 다섯 번 안에 세 번 성공하면 확정인데, 여기서는 네 번째 스캔에서 세 번째 성공이 나와서 그 자리에서 확정됩니다. 반대로 M 번 연속으로 실패하면 지웁니다.

아래 확정 트랙은 연관에 실패하는 순간 메모리 트랙으로 바뀌고, 메모리 상태가 정해 둔 시간보다 길어지면 표적을 놓친 것으로 보고 지웁니다.

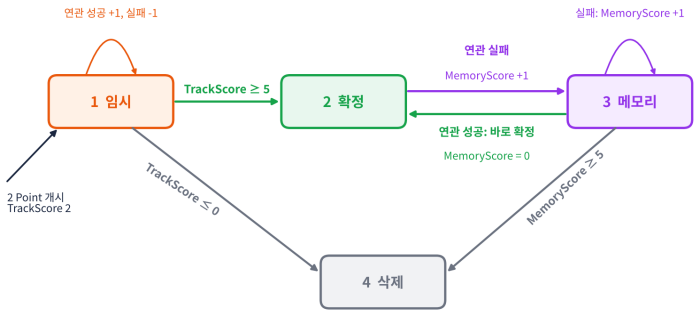
횟수만 세면 되니까 단순하다는 게 장점인데, 최근 N 번의 성공, 실패 기록을 계속 들고 있어야 합니다. 이번 과제에서는 과제 조건대로 다음 장의 트랙 스코어 방식을 썼습니다.

# 13 2.3 트랙 스코어 방식

02. 트랙 관리 과제 2.2 · 3-2

## 2.3 트랙 스코어 방식 (이번 과제)

트랙마다 점수 두 개 (TrackScore, MemoryScore) 를 두고, 연관 성공 · 실패에 따라 점수를 올리고 내리다가 문턱을 넘으면 상태를 바꿉니다.




1. **TrackState (과제 값)**
   1 임시, 2 확정, 3 메모리, 4 삭제
2. **문턱 (과제 값)**
   임시 트랙 확정 5, 임시 트랙 삭제 0
   메모리 트랙 삭제 5
3. **메모리 트랙이 다시 연관되면**
   바로 확정 트랙으로 되돌리고 MemoryScore = 0
4. **과제에 없어 정한 값**
   개시 직후 TrackScore = 2 (플롯 두 개가 이미 이어졌으므로)
   한 번에 올리고 내리는 양 = 1

M/N 과 달리 정수 두 개만 있으면 같은 판단을 할 수 있습니다.

13

### 발표 대본

트랙 관리 · 약 60초

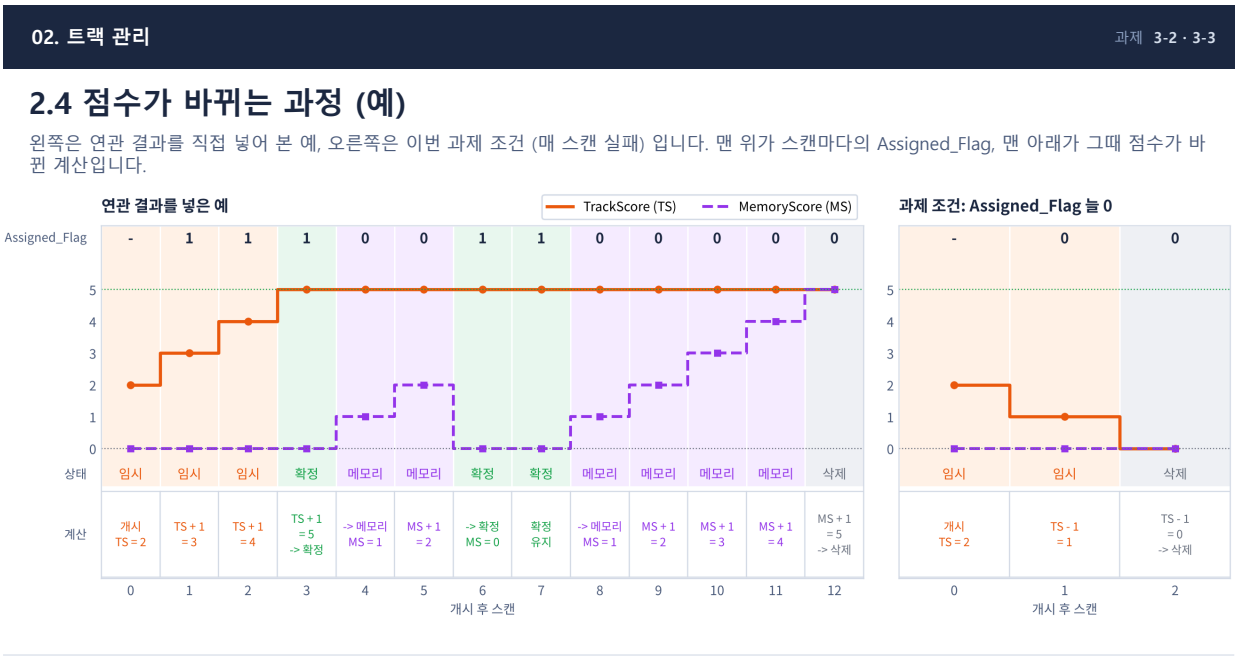
이번 과제에서 쓴 트랙 스코어 방식입니다. 트랙마다 TrackScore 와 MemoryScore 두 점수를 들고 다니면서, 점수가 문턱을 넘으면 상태를 바꿉니다. 상태 번호는 과제에서 정한 대로 1 임시, 2 확정, 3 메모리, 4 삭제입니다.

그림 왼쪽부터 보면, 임시 트랙은 연관에 성공하면 TrackScore 를 1 올리고 실패하면 1 내립니다. 그래서 5 이상이 되면 확정, 0 이하가 되면 삭제입니다.

확정 트랙이 연관에 실패하면 메모리 트랙으로 바꾸고 MemoryScore 를 1 올립니다. 메모리 트랙이 다시 연관되면 바로 확정으로 돌아가면서 MemoryScore 를 0 으로 되돌리고, 계속 실패해서 MemoryScore 가 5 가 되면 삭제합니다.

문턱 값 5, 0, 5 는 과제에서 준 값이고, 과제에 없는 값 두 개는 제가 정했습니다. 두 플롯으로 만든 트랙이라 개시 직후 TrackScore 는 2 로 두었고, 한 번에 올리고 내리는 양은 1 로 했습니다.

# 2.4 점수가 바뀌는 과정 (예)




## 발표 대본

트랙 관리 · 약 65초

점수가 실제로 어떻게 움직이는지 두 가지 경우로 보겠습니다. 맨 위 숫자가 스캔마다 넣은 Assigned_Flag 이고, 맨 아래 계산 줄에 그 스캔에서 점수가 어떻게 바뀌었는지 적어 두었습니다.

왼쪽은 연관 결과를 직접 넣어 본 예입니다. 2 에서 시작한 TrackScore 가 성공할 때마다 1 씩 올라서 5 가 되는 3 번 스캔에 확정됩니다. 4 번 스캔에서 실패하면 메모리로 바뀌면서 MemoryScore 가 1 이 되고, 한 번 더 실패하면 2 가 됩니다. 6 번 스캔에서 다시 성공하면 바로 확정으로 돌아오고 MemoryScore 는 0 이 됩니다. 마지막에 다섯 번 연속으로 실패하면 MemoryScore 가 5 가 되는 12 번 스캔에서 삭제됩니다.

오른쪽이 이번 과제 조건입니다. 매 스캔 0 이 들어가니까 TrackScore 가 2 에서 1, 0 으로 내려가서 세 번째 스캔에 삭제됩니다. 그래서 확정이나 메모리 상태는 시뮬레이션에서 나오지 않고, 이건 4.2 에서 따로 확인했습니다.

## 03. 구현

쓰레드 · UDP 구조와 플롯 메시지를 보고, 송신 · 수신 코드와 트랙 개시 · 관리 코드를 차례로 봅니다.


**3.1** 전체 구조: 쓰레드와 UDP ..... 16

**3.2** 플롯 메시지 STRUCT_PlotMsg ..... 17

**3.3** 송신: 플롯 메시지 만들기 ..... 18

**3.4** 수신: 좌표 변환과 트랙 처리 ..... 19

**3.5** 2-Point 개시: 쌍 고르기 ..... 20

**3.6** 2-Point 초기화 ..... 21

**3.7** 트랙 스코어 로직 ..... 22



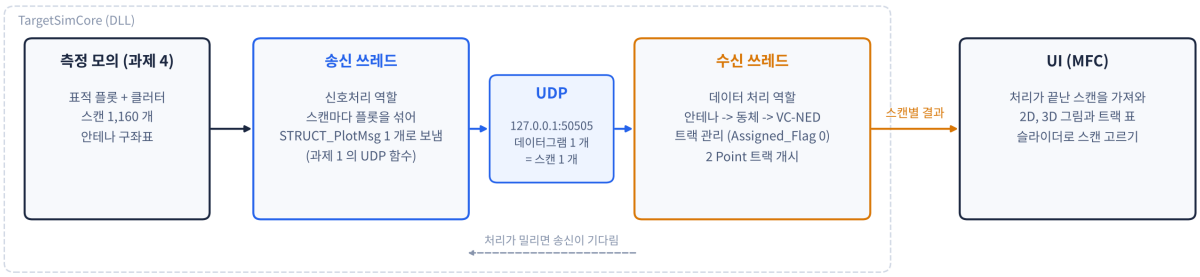

**발표 대본** 구현 · 약 10초

세 번째로 구현입니다. 쓰레드와 UDP 구조, 플롯 메시지를 먼저 보고, 송신, 수신, 트랙 개시, 스코어 로직 코드를 차례로 보겠습니다.

03. 구현 과제 3-3

## 3.1 전체 구조: 쓰레드와 UDP

과제 4 의 측정 결과를 신호처리가 보낸 플롯이라고 보고, 송신 쓰레드가 스캔마다 UDP 로 보내면 수신 쓰레드가 받아 트랙을 처리합니다.




**왜 TCP 가 아니라 UDP 인가** 과제는 UDP 또는 TCP/IP 중 하나를 고르게 함

| | UDP (이번 과제) | TCP |
|---|---|---|
| 메시지 단위 | 보낸 메시지 1 개 = 받는 메시지 1 개 (스캔 1 개) | 바이트가 이어서 와서, 메시지를 직접 잘라야 함 |
| 지연 | 연결 · 재전송 없이 바로 전달 | 하나가 늦으면 뒤 스캔도 함께 기다림 |
| 손실 | 생길 수 있음 -> 스캔 번호로 확인 (1,160 / 1,160, 빠짐 0) | 없음 (잃으면 다시 보냄) |

16

### 발표 대본

구현 · 약 65초

전체 구조부터 보겠습니다. 과제 4 의 측정 모의 결과를 레이다 신호처리가 내보낸 플롯이라고 보고, 데이터 처리 쪽이 그것을 받아서 처리하는 모양으로 만들었습니다.

송신 쓰레드가 신호처리 역할입니다. 스캔마다 표적 플롯과 클러터를 섞어서 메시지 하나를 만들고 UDP 로 보냅니다. 수신 쓰레드는 데이터 처리 역할인데요, 메시지를 받으면 플롯 좌표를 VC-NED 로 바꾸고 트랙 관리와 2-Point 개시를 돌린 다음 결과를 스캔 단위로 쌓아 둡니다. UI 는 그 결과를 가져다 그림과 표로 보여 줍니다.

아래 표는 TCP 가 아니라 UDP 를 고른 이유입니다. UDP 는 보낸 메시지 하나가 그대로 하나로 도착해서 스캔 하나를 한 번에 받을 수 있고, 연결이나 재전송 없이 바로 전달됩니다. TCP 는 바이트가 이어서 오기 때문에 메시지를 직접 잘라야 하고, 하나가 늦으면 뒤 스캔도 같이 기다리게 됩니다. UDP 의 약점은 손실인데, 스캔 번호로 빠진 게 없는지 확인했고 시험에서 1,160 스캔을 모두 받았습니다.




## 발표 대본

구현 · 약 60초

다음은 실제로 주고받는 플롯 메시지입니다. 구조체를 코드로 보면 눈에 잘 안 들어와서 그림으로 그렸습니다.

왼쪽 파란 상자가 메시지 머리입니다. 메시지 종류와 크기, 스캔 번호와 시각, 플롯 수가 들어가고, 측정 표준편차와 플랫폼 위치, 자세도 같이 보냅니다. 받는 쪽에서 오차 공분산을 계산하고 좌표를 바꿀 때 필요하기 때문입니다.

가운데 주황 상자가 플롯 배열입니다. 플롯 하나마다 칸이 이렇게 나뉘는데, 빨간 칸은 보내는 쪽이 채우는 것이고 파란 칸은 받는 쪽이 계산해서 채우는 것입니다. 과제 조건대로 보내는 쪽은 안테나 기준 거리, 방위각, 고각 세 칸과 ID 같은 탐지 정보만 채웁니다. 표적 플롯과 클러터가 섞여 있고 어느 게 표적인지 표시는 없습니다.

보낼 때는 플롯 수만큼만 잘라서 보내고, 참고 소스 코드의 Idd.h 가 STRUCT_PlotInfo 의 Y_Body 에서 끊겨 있어서 그 뒤를 이어 완성했습니다.




## 발표 대본

구현 · 약 55초

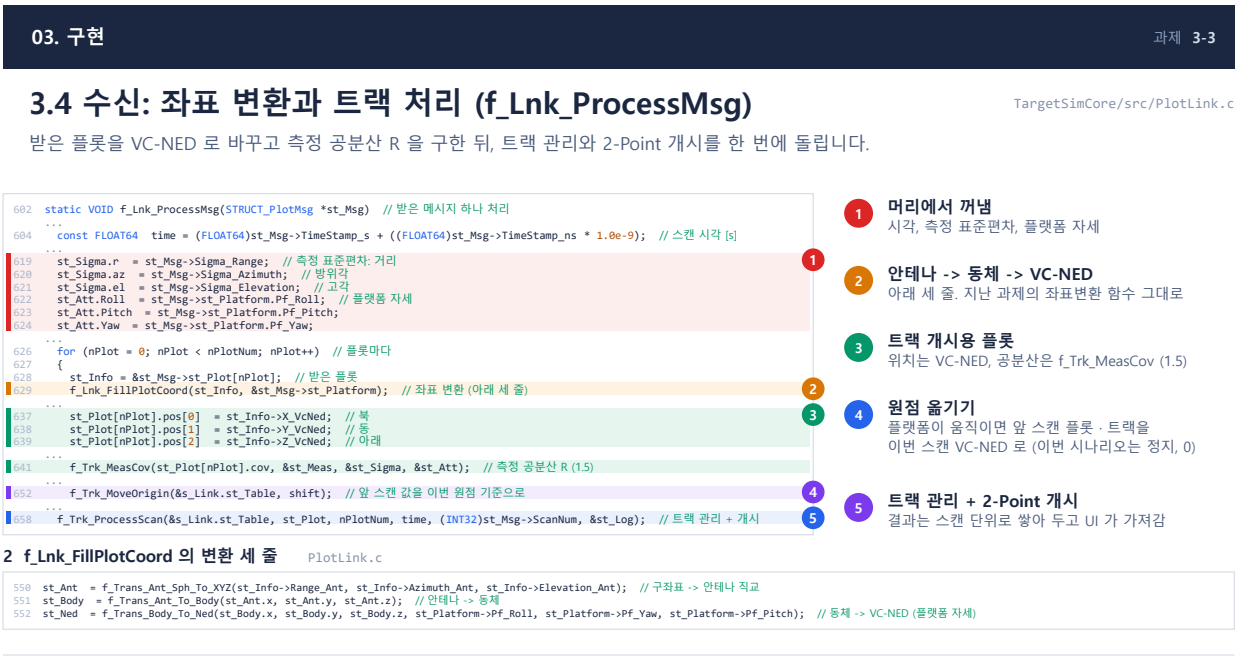
송신 쪽에서 메시지를 만드는 f_Lnk_BuildMsg 입니다. 코드 오른쪽 주석과 번호를 같이 보시면 됩니다.

먼저 그 스캔의 표적 플롯을 배열에 담고, 이어서 클러터를 같은 배열에 붙입니다. 이때 각 플롯이 어느 표적에서 나왔는지는 송신 쪽 기록에만 따로 적어 둡니다. 그다음 방위각 순서로 정렬하는데, 빔이 지나가는 순서대로 플롯이 나오는 것처럼 보이게 하려는 것이고, 동시에 배열 위치만 보고는 표적과 클러터를 구분할 수 없게 하려는 것입니다.

다음으로 메시지 머리에 스캔 번호, 시각, 플롯 수를 채우고, 플롯마다 ID 를 붙이면서 안테나 기준 거리, 방위각, 고각을 넣습니다. ID 는 과제 조건대로 10000 부터 하나씩 올리고, 1000000 다음에는 다시 10000 으로 돌아갑니다.

이렇게 만든 메시지를 송신 쓰레드가 과제 1 의 UDP 송신 함수로 보냅니다.

# 3.4 수신: 좌표 변환과 트랙 처리




## 발표 대본

구현 · 약 65초

이번에는 받는 쪽입니다. 데이터 처리의 중심인 f_Lnk_ProcessMsg 를 보겠습니다.

먼저 메시지 머리에서 시각, 측정 표준편차, 플랫폼 자세를 꺼냅니다.

그다음 플롯마다 f_Lnk_FillPlotCoord 를 불러 좌표를 바꿉니다. 아래 세 줄처럼 안테나 구좌표를 안테나 직교좌표로, 다시 동체로, 플랫폼 자세를 써서 VC-NED 로 옮기는데, 변환 함수는 지난 과제들에서 쓰던 좌표 변환 코드 그대로입니다. 그렇게 나온 VC-NED 위치와, 1.5 에서 본 식으로 계산한 측정 공분산을 묶어서 트랙 개시용 플롯을 만듭니다.

여기서 하나 짚고 가면, VC-NED 는 원점이 플랫폼을 따라 움직입니다. 그래서 플랫폼이 움직이면 앞 스캔과 이번 스캔의 원점이 달라지는데, f_Trk_MoveOrigin 으로 앞 스캔 값을 이번 원점 기준으로 옮겨 두면 두 플롯의 차가 상대 속도가 아니라 표적 속도가 됩니다. 이번 시나리오는 플랫폼이 서 있어서 옮기는 양은 0 입니다.

마지막으로 f_Trk_ProcessScan 을 불러 트랙 관리와 개시를 한 번에 돌립니다.




## 발표 대본

구현 · 약 65초

스캔 하나를 처리하는 f_Trk_ProcessScan 입니다. 코드는 크게 두 부분으로 나뉘는데, 첫 번째가 트랙 관리입니다. 연관 기능이 아직 없으니까 모든 트랙의 Assigned_Flag 를 0 으로 두고 스코어 로직을 돌립니다.

두 번째가 2-Point 개시입니다. 앞 스캔에서 아직 안 쓴 플롯과 이번 스캔 플롯을 모두 짝지어 속력을 구하고, 그중 가장 느린 쌍을 찾습니다. 여기서 눈여겨보실 부분은 bestSpeed 를 V_max 로 시작한다는 점인데요, 이렇게 하면 800 m/s 이상인 쌍은 처음부터 후보가 될 수 없어서 이 한 줄이 속도 게이트 역할을 합니다.

쌍을 고르면 두 플롯 모두 썼다고 표시해서 다른 쌍에 다시 쓰이지 않게 하고, 새 ID 를 받아 트랙을 만듭니다. 상태는 임시, TrackScore 는 2 로 시작하고, 개시에 쓴 두 플롯의 ID 도 남겨 둡니다. 이 과정을 고를 쌍이 없을 때까지 반복합니다. 트랙 ID 는 100 부터 차례로 주고, 10000 다음에는 다시 100 으로 돌아가면서 쓰고 있는 번호는 건너뜁니다.




## 발표 대본

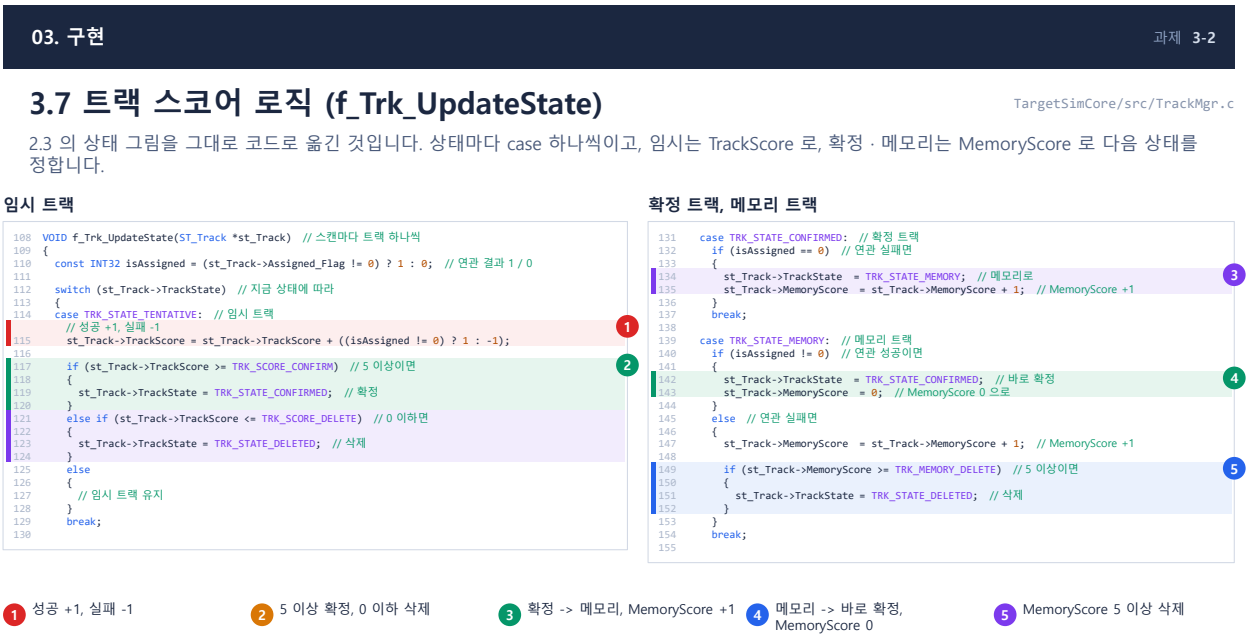
구현 · 약 50초

트랙의 초기 상태변수와 오차 공분산을 만드는 f_Trk_TwoPointInit 입니다.

먼저 참고 소스 코드의 행렬 함수로 9 x 1 상태변수와 9 x 9 오차 공분산을 만들면서 전부 0 으로 채웁니다. 그래서 가속도 칸은 따로 건드리지 않아도 0 으로 남습니다.

바깥 반복문은 축마다 돌면서 위치에는 이번 플롯 위치를, 속도에는 두 플롯 위치 차를 시간 차로 나눈 값을 넣습니다.

안쪽 반복문이 오차 공분산을 채우는 부분입니다. 축 a 와 축 b 의 3 x 3 칸마다 위치는 R_k, 위치와 속도 사이는 R_k 나누기 T, 속도는 R_k 더하기 R_k-1 을 T 제곱으로 나눈 값을 넣습니다. 1.3 에서 본 3 x 3 표 그대로입니다. 코드의 3 곱하기 nAxis 는 축 a 의 위치 칸, 거기에 1 을 더하면 속도 칸이라는 뜻입니다.



03. 구현 과제 3-2

## 3.7 트랙 스코어 로직 (f_Trk_UpdateState)

TargetSimCore/src/TrackMgr.c

2.3 의 상태 그림을 그대로 코드로 옮긴 것입니다. 상태마다 case 하나씩이고, 임시는 TrackScore 로, 확정 · 메모리는 MemoryScore 로 다음 상태를 정합니다.

**임시 트랙**

```
VOID f_Trk_UpdateState(ST_Track *st_Track)  // 스캔마다 트랙 하나씩
{
  const INT32 isAssigned = (st_Track->Assigned_Flag != 0) ? 1 : 0;  // 연관 결과 1 / 0

  switch (st_Track->TrackState)  // 지금 상태에 따라
  {
  case TRK_STATE_TENTATIVE:  // 임시 트랙
    // 성공 +1, 실패 -1
    st_Track->TrackScore = st_Track->TrackScore + ((isAssigned != 0) ? 1 : -1);

    if (st_Track->TrackScore >= TRK_SCORE_CONFIRM)  // 5 이상이면
    {
      st_Track->TrackState = TRK_STATE_CONFIRMED;  // 확정
    }
    else if (st_Track->TrackScore <= TRK_SCORE_DELETE)  // 0 이하면
    {
      st_Track->TrackState = TRK_STATE_DELETED;  // 삭제
    }
    else
    {
      // 임시 트랙 유지
    }
    break;
```

**확정 트랙, 메모리 트랙**

```
  case TRK_STATE_CONFIRMED:  // 확정 트랙
    if (isAssigned == 0)  // 연관 실패면
    {
      st_Track->TrackState  = TRK_STATE_MEMORY;  // 메모리로
      st_Track->MemoryScore  = st_Track->MemoryScore + 1;  // MemoryScore +1
    }
    break;

  case TRK_STATE_MEMORY:  // 메모리 트랙
    if (isAssigned != 0)  // 연관 성공이면
    {
      st_Track->TrackState  = TRK_STATE_CONFIRMED;  // 바로 확정
      st_Track->MemoryScore  = 0;  // MemoryScore 0 으로
    }
    else  // 연관 실패면
    {
      st_Track->MemoryScore  = st_Track->MemoryScore + 1;  // MemoryScore +1

      if (st_Track->MemoryScore >= TRK_MEMORY_DELETE)  // 5 이상이면
      {
        st_Track->TrackState = TRK_STATE_DELETED;  // 삭제
      }
    }
    break;
```

1 성공 +1, 실패 -1 2 5 이상 확정, 0 이하 삭제 3 확정 -> 메모리, MemoryScore +1 4 메모리 -> 바로 확정, MemoryScore 0 5 MemoryScore 5 이상 삭제

22

### 발표 대본

구현 · 약 55초

트랙 스코어 로직인 f_Trk_UpdateState 입니다. 지금 상태에 따라 switch 의 case 가 하나씩 있어서, 2.3 에서 본 상태 그림을 그대로 코드로 옮겼다고 보시면 됩니다.

왼쪽의 임시 트랙은 Assigned_Flag 가 1 이면 TrackScore 를 1 올리고 0 이면 1 내린 다음, 5 이상이면 확정, 0 이하면 삭제로 바꿉니다.

오른쪽 위의 확정 트랙은 연관에 실패하면 메모리로 바꾸면서 MemoryScore 를 1 올립니다.

메모리 트랙은 연관에 성공하면 바로 확정으로 돌리고 MemoryScore 를 0 으로 되돌립니다. 실패하면 MemoryScore 를 1 올리고, 5 이상이 되면 삭제로 바꿉니다.

삭제 상태가 된 트랙은 다음 스캔을 시작할 때 트랙 표에서 빠집니다.

## 04. 결과

시뮬레이션 조건과 2D · 3D 결과 화면, 트랙이 생기고 사라지는 과정, 거리에 따라 개시 결과가 어떻게 달라지는지 봅니다.


**4.1** 시뮬레이션 조건과 결과 화면 (True · Plot · Track) ..... 24

**4.2** 트랙 수명과 스코어 확인 ..... 25

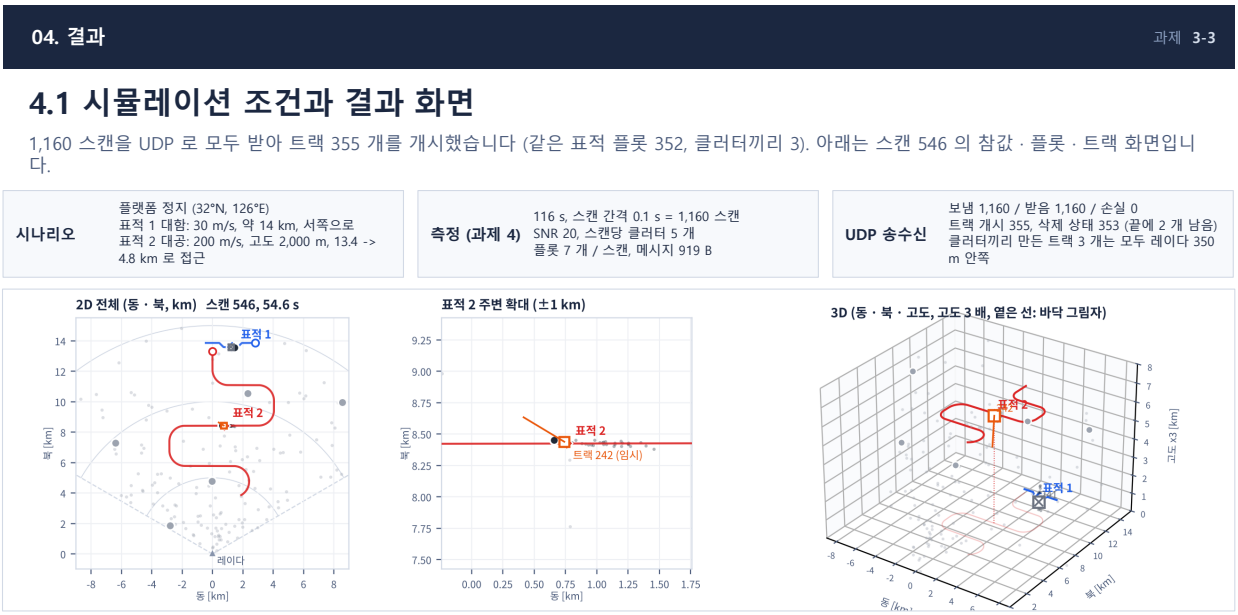
**4.3** 거리에 따른 개시 결과 ..... 26




**발표 대본** 결과 · 약 10초

마지막으로 결과입니다. 시뮬레이션 조건과 2D, 3D 화면, 트랙이 생기고 사라지는 과정과 스코어 확인, 거리에 따라 개시 결과가 어떻게 달라지는지까지 보겠습니다.

# 4.1 시뮬레이션 조건과 결과 화면




## 발표 대본

결과 · 약 80초

먼저 조건부터 말씀드리면, 위쪽 상자 세 개가 시나리오, 측정, 송수신 조건입니다. 플랫폼은 정지해 있고, 표적 1 은 약 14 km 에서 서쪽으로 천천히 가는 대함 표적, 표적 2 는 고도 2,000 m 에서 13.4 km 부터 4.8 km 까지 지그재그로 다가오는 대공 표적입니다. 116 초를 0.1 초 간격으로 재서 1,160 스캔이고, 스캔마다 표적 플롯 2 개에 클러터 5 개를 섞어 플롯 7 개짜리 메시지를 보냈습니다. UDP 로 1,160 개를 손실 없이 받아서 트랙 355 개를 개시했습니다. 352 개는 두 플롯이 같은 표적에서 나왔고, 클러터끼리 짝지어진 3 개는 모두 클러터가 몰려 있는 레이다 350 m 안쪽이었습니다.

아래는 과제에서 요구한 참값, 플롯, 트랙 심볼을 함께 그린 546 번 스캔 그림으로, 프로그램이 낸 결과 자료로 다시 그린 것입니다. 왼쪽 2D 에서 파란 선이 표적 1, 빨간 선이 표적 2 의 참값 궤적이고, 큰 점이 지금 스캔에 받은 플롯, 옅은 점이 지난 30 스캔의 플롯입니다. 빈 주황 사각형이 임시 트랙, 회색 X 가 삭제 상태가 된 트랙입니다. 가운데는 표적 2 주변을 확대한 것인데, 사각형에서 뻗은 선은 트랙 속도로 1 초 동안 가는 거리입니다. 위치 오차를 0.1 초로 나눠 구한 속도라 궤적 방향과 꽤 어긋나는 것이 보입니다.

오른쪽 3D 는 고도를 3 배로 그렸고, 옅은 선이 바닥 그림자라 선과 그림자 사이가 고도입니다. 표적 2 는 같은 고도로 떠서 날고, 표적 1 은 바닥에 붙어 있어서 두 표적이 높이로도 갈려 보입니다.

04. 결과 과제 3-2 · 3-3

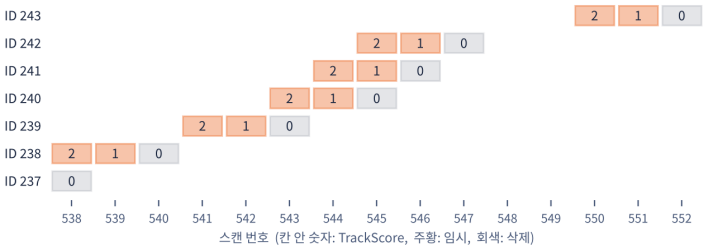
## 4.2 트랙 수명과 스코어 확인

연관이 늘 실패인 과제 조건에서는 모든 트랙이 TrackScore 2 -> 1 -> 0 으로 내려가 세 스캔 만에 삭제됩니다. 확정 · 메모리는 스코어 확인 창으로 검증했습니다.

**스캔 545 의 트랙 표** 프로그램 결과 표의 값

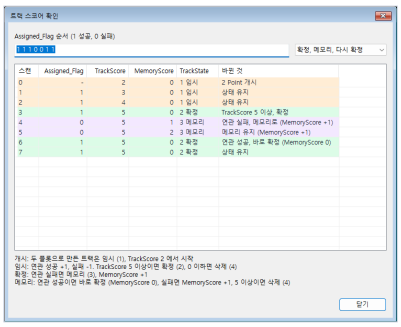
| 트랙 ID | TrackState | TrackScore | MemoryScore | Assigned_Flag | 개시 스캔 | 개시 플롯 (앞, 이번) | 참값 |
|---|---|---|---|---|---|---|---|
| 240 | 4 삭제 | 0 | 0 | 0 | 543 | 13796, 13804 | 표적 2 |
| 241 | 1 임시 | 1 | 0 | 0 | 544 | 13805, 13814 | 표적 1 |
| 242 | 1 임시 | 2 | 0 | 0 | 545 | 13813, 13819 | 표적 2 |

**1 한 스캔에 나이가 다른 트랙 세 개**
242: 이번 스캔에 개시, TrackScore 2 / 241: 한 스캔 전 개시, 1
240: 두 스캔 전 개시, 0 -> 삭제 (4). 다음 스캔에 표에서 빠짐



스캔 번호 (칸 안 숫자: TrackScore, 주황: 임시, 회색: 삭제)

**스코어 확인 창** 연관 결과를 직접 넣어 봄



**2 1 1 1 0 0 1 1 을 넣으면**
3 번째 스캔에 확정, 두 번 실패해 메모리, 다시 성공하면 바로 확정 (MemoryScore 0). 같은 f_Trk_UpdateState 를 부름

### 발표 대본

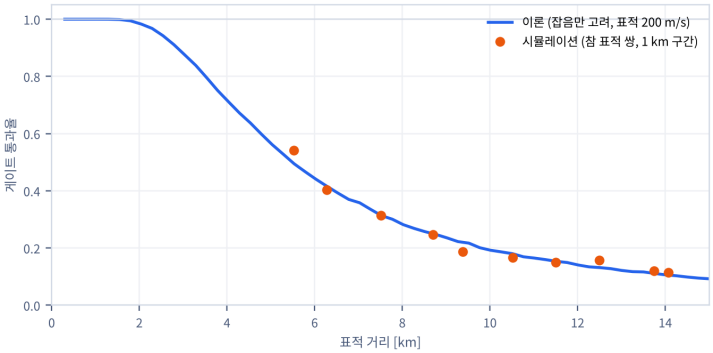
결과 · 약 60초

왼쪽 위는 545 번 스캔의 트랙 표입니다. 한 스캔에 나이가 다른 트랙 세 개가 보이는데요, 242 번은 이번 스캔에 만들어져서 TrackScore 가 2, 241 번은 한 스캔 전에 만들어져서 1, 240 번은 두 스캔 전에 만들어져서 0 이 되면서 삭제 상태가 됐습니다. 아래 띠 그림은 538 번부터 552 번 스캔까지 트랙마다 점수가 어떻게 내려가는지 그린 것인데, 모든 트랙이 2, 1, 0 세 칸을 거치고 사라지는 걸 볼 수 있습니다. 과제 조건에서는 연관이 늘 실패니까 당연한 결과입니다.

그러다 보니 확정이나 메모리 상태는 시뮬레이션에서는 볼 수가 없어서, 오른쪽의 스코어 확인 창을 따로 만들었습니다. 연관 결과를 직접 입력하면 같은 스코어 함수를 그대로 불러서 스캔마다 점수와 상태를 보여줍니다. 1 1 1 0 0 1 1 을 넣으면 세 번째 스캔에 확정되고, 두 번 실패해서 메모리로 갔다가, 다시 성공하면서 MemoryScore 가 0 으로 돌아가고 확정되는 것을 확인할 수 있습니다.

## 4.3 거리에 따른 개시 결과

멀리 있는 표적일수록 플롯이 크게 흔들려 같은 표적의 두 플롯도 80 m 게이트를 넘기 쉽습니다. 그래서 먼 표적은 트랙이 덜 만들어집니다.




| 표적 | 거리 [km] | 스캔 쌍 | 게이트 통과 | 개시 트랙 | 개시 비율 |
|---|---|---|---|---|---|
| 2 | 4.8 ~ 6.5 | 305 | 45 % | 96 | 31 % |
| 2 | 6.5 ~ 8.0 | 87 | 32 % | 21 | 24 % |
| 2 | 8.0 ~ 10.5 | 414 | 23 % | 71 | 17 % |
| 2 | 10.5 ~ 13.4 | 348 | 15 % | 46 | 13 % |
| 1 | 13.6 ~ 14.2 | 1,159 | 12 % | 117 | 10 % |

**읽는 법**
왼쪽 파란 선은 측정 잡음만으로 계산한 이론 통과율, 주황 점은 시뮬레이션입니다.
표는 가까운 순서입니다. 표적 2 는 다가오는 표적이라 거리 구간으로 나눴고, 개시 비율 = 개시 트랙 / 스캔 쌍 입니다.
각도 오차 0.48° 가 거리를 곱한 만큼 옆 방향 위치 오차가 됩니다. 5 km 면 ±42 m, 14 km 면 ±117 m.



### 발표 대본

결과 · 약 60초

마지막으로 거리에 따라 트랙이 얼마나 만들어졌는지 보겠습니다.

왼쪽 그래프는 같은 표적의 플롯 쌍이 속도 게이트를 통과한 비율을 거리별로 그린 것입니다. 파란 선이 측정 잡음으로 계산한 이론값, 주황 점이 시뮬레이션인데 둘이 잘 맞습니다. 2 km 안쪽은 거의 다 통과하는데, 5 km 에서 절반 남짓, 14 km 에서는 열 번에 한 번 정도로 떨어집니다. 1.4 에서 본 것처럼 멀수록 플롯이 크게 흔들려서 80 m 게이트를 넘기 쉽기 때문입니다.

오른쪽 표는 가까운 순서로 놓은 것입니다. 표적 2 는 13.4 km 에서 4.8 km 까지 다가오는 표적이라 거리 구간으로 나눴고, 구간마다 머문 스캔 쌍 수가 달라서 맨 오른쪽에 개시 비율을 함께 적었습니다. 가장 가까운 구간은 세 번에 한 번꼴로 트랙이 만들어졌고, 14 km 의 표적 1 은 열 번에 한 번꼴이라 멀수록 트랙이 덜 만들어지는 걸 확인할 수 있습니다.

# 27 정리

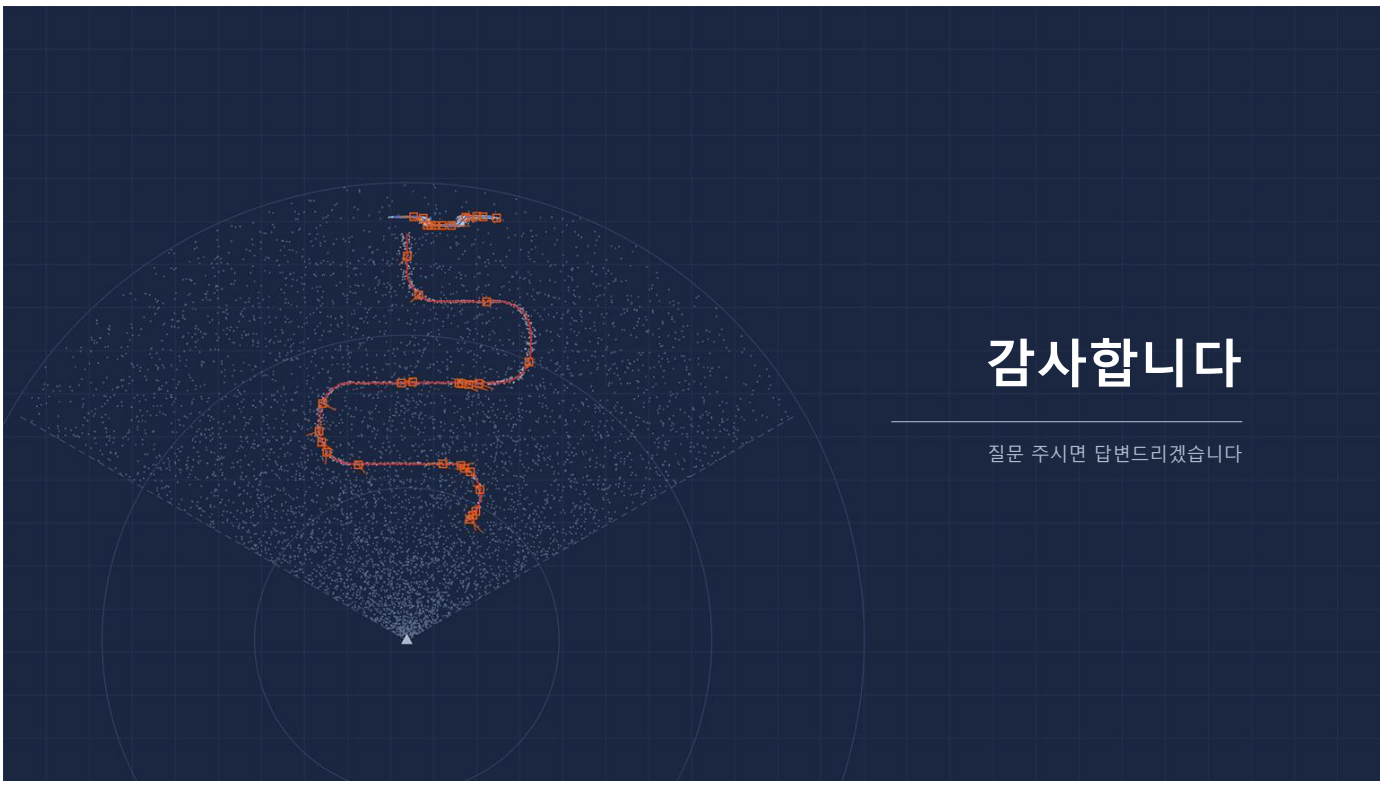



## 발표 대본

정리 · 약 40초

정리하겠습니다. 트랙 개시는 연속 두 스캔의 플롯 쌍 가운데 속도 게이트를 통과한 쌍으로 2-Point 개시를 했고, 상태변수 9 개와 9 x 9 오차 공분산, 트랙 ID 100 부터 10000 까지 과제 조건을 그대로 따랐습니다. 트랙 관리는 TrackScore 와 MemoryScore 두 점수와 문턱 5, 0, 5 로 네 가지 상태를 오가게 했습니다. 시뮬레이션은 플롯 메시지를 UDP 로 스캔마다 주고받아 1,160 스캔을 손실 없이 처리했고, 2D 와 3D 에 참값, 플롯, 트랙을 함께 그렸습니다.

연관이 늘 실패인 조건이라 트랙이 세 스캔 뒤 지워지는 건 확인했고, 확정과 메모리 로직은 스코어 확인 창으로 따로 검증했습니다. 다음 과제에서 트랙 연관과 추적 필터가 붙으면 이번에 만든 확정, 메모리 로직이 실제 시뮬레이션 안에서 동작하게 됩니다.




**발표 대본** 마무리 · 약 10초

이상으로 발표를 마치겠습니다. 들어 주셔서 감사합니다. 궁금한 점 있으시면 편하게 질문 주세요.